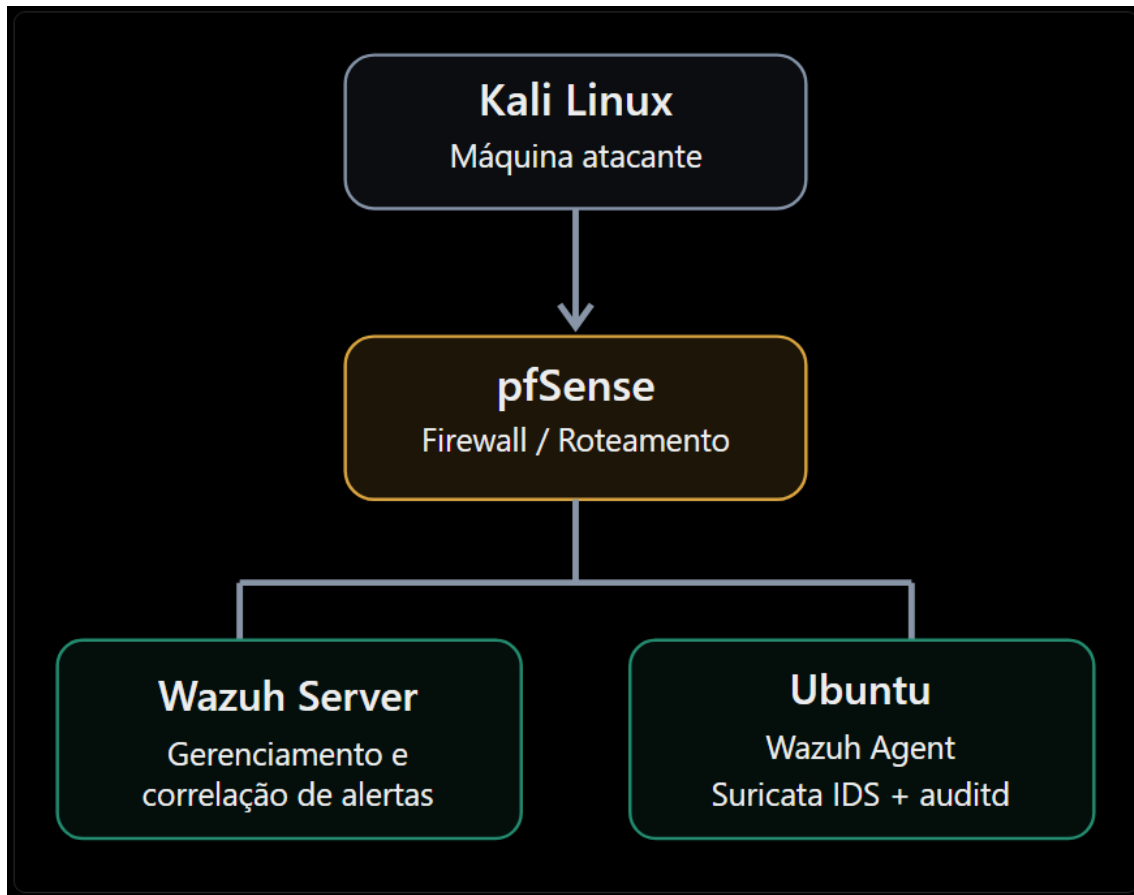


Ambiente SOC com Wazuh



Depois de instalar o servidor Wazuh e abrir sua interface em https://<ip_servidor_wazuh>, é necessário adicionar seus agentes indo em “Agents Management” > “Summary” > “Deploy new agente”, depois disso é só seguir as instruções e baixar o agente no cliente desejado.

Agents (2)

Deploy new agent

Refresh

Export formatted

More

Search

WQL

☐	ID ↑	Name	IP address	Group(s)	Operating system	Cluster node	Version	Status	Actions
☐	001	WinSOC	192.168.10.20	default	 Microsoft Windows 10 Pro 10.0.19045.6456	node01	v4.14.8	active	
☐	005	ubuntu	192.168.10.50	default	 Ubuntu 26.04.1 LTS	node01	v4.14.8	active	

Rows per page: 10

<

1

>

1) Monitoramento de Integridade de Arquivos

O Wazuh permite monitorar alterações em arquivos e diretórios por meio do File Integrity Monitoring (FIM), identificando eventos como criação, modificação e exclusão de arquivos.

Para configurar esse monitoramento, é necessário ajustar o arquivo `ossec.conf`, localizado em `/var/ossec/etc/`, de acordo com o que for ser monitorado.

Esse é o arquivo principal de configuração do Wazuh, responsável por definir quais recursos de monitoramento estão habilitados e como eles funcionam.

```
<ossec_config>
  <global>
    <jsonout_output>yes</jsonout_output>
    <alerts_log>yes</alerts_log>
    <logall>yes</logall>
    <logall_json>no</logall_json>
    <email_notification>no</email_notification>
    <smtp_server>smtp.example.wazuh.com</smtp_server>
    <email_from>wazuh@example.wazuh.com</email_from>
    <email_to>recipient@example.wazuh.com</email_to>
    <email_maxperhour>12</email_maxperhour>
    <email_log_source>alerts.log</email_log_source>
    <agents_disconnection_time>15m</agents_disconnection_time>
    <agents_disconnection_alert_time>0</agents_disconnection_alert_time>
    <update_check>yes</update_check>
  </global>
```

Ao definir a opção “logall” como “yes” no arquivo ossec.conf do Wazuh Manager, habilitamos o arquivamento de todos os eventos recebidos, incluindo aqueles que não geram alertas.

Após a alteração, é necessário reiniciar o serviço wazuh-manager para aplicar a configuração.

Isso pode ser feito com “**systemctl restart wazuh-manager**”.

Para configurar o File Integrity Monitoring (FIM), é necessário editar o arquivo /var/ossec/etc/ossec.conf no agente Wazuh instalado no Ubuntu.

```
<!-- File integrity monitoring -->
<syscheck>
  <disabled>no</disabled>

  <!-- Frequency that syscheck is executed default every 12 hours -->
  <frequency>43200</frequency>

  <scan_on_start>yes</scan_on_start>

  <!-- Directories to check (perform all possible verifications) -->
  <directories>/etc,/usr/bin,/usr/sbin</directories>
  <directories>/bin,/sbin,/boot</directories>

  <!-- Files/directories to ignore -->
  <ignore>/etc/mtab</ignore>
  <ignore>/etc/hosts.deny</ignore>
  <ignore>/etc/mail/statistics</ignore>
  <ignore>/etc/random.seed</ignore>
  <ignore>/etc/random.seed</ignore>
  <ignore>/etc/adftime</ignore>
  <ignore>/etc/httpd/logs</ignore>
  <ignore>/etc/utmpx</ignore>
  <ignore>/etc/wtmpx</ignore>
  <ignore>/etc/cups/certs</ignore>
  <ignore>/etc/dumpdates</ignore>
  <ignore>/etc/svc/volatile</ignore>

  <!-- File types to ignore -->
  <ignore type="sregex">.log$.swp$</ignore>

  <!-- Check the file, but never compute the diff -->
  <nodiff>/etc/ssl/private.key</nodiff>

  <skip_nfs>yes</skip_nfs>
  <skip_dev>yes</skip_dev>
```

Na seção <!-- File Integrity Monitoring -->, é possível configurar os diretórios que serão monitorados em <directories>.

```
<!-- Directories to check (perform all possible verifications) -->
<directories>/etc,/usr/bin,/usr/sbin</directories>
<directories>/bin,/sbin,/boot</directories>
<directories check_all="yes" report_changes="yes" realtime="yes">/root</directories>
<!-- Files/directories to ignore -->
```

Neste laboratório, foi escolhido o diretório /root, com o objetivo de detectar alterações em arquivos e gerar alertas para análise no Wazuh.

- check_all="yes": para checar tudo
- report_changes="yes": reporta qualquer mudança
- realtime="yes": habilita o monitoramento em tempo real

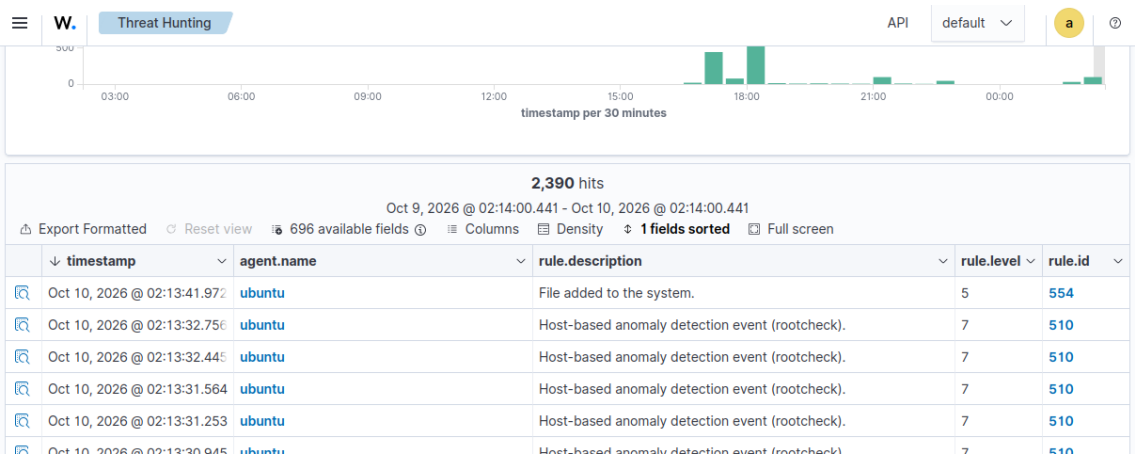
Essa configuração pode gerar um grande volume de eventos, mas no caso de um ambiente de testes é aceitável.

```
root@thiago-VMware-Virtual-Platform:/home/thiago# systemctl restart wazuh-agent
root@thiago-VMware-Virtual-Platform:/home/thiago#
```

Após a configuração, é necessário reiniciar o serviço wazuh-agent para aplicar as alterações.

Para validar o funcionamento do FIM, foi realizado um teste com a criação do arquivo malware.txt no diretório /root.

```
root@thiago-VMware-Virtual-Platform:~# touch malware.txt
root@thiago-VMware-Virtual-Platform:~# pwd
/root
```



Após a criação do arquivo, foi acessada a seção Threat Hunting > Events no dashboard do Wazuh para verificar se o evento havia sido detectado.

O Wazuh gerou um alerta relacionado à criação do arquivo monitorado. Ao expandir os detalhes do evento, é possível identificar informações importantes para a investigação, como o agente responsável, o caminho do arquivo, a ação detectada e a regra acionada.

Document Details

[View surrounding documents](#)[View single document](#)

Table JSON

t _index	wazuh-alerts-4.x-2026.10.10
t agent.id	005
t agent.ip	192.168.10.50
t agent.name	ubuntu
t decoder.name	syscheck_new_entry
t full_log	File '/root/malware.txt' added Mode: realtime
t id	1791609221.444397
t input.type	log
t location	syscheck
t manager.name	wazuh-server
t rule.description	File added to the system.
# rule.firedtimes	1
t rule.gdpr	II_5.1.f
t rule.gpg13	4.11
t rule.groups	ossec, syscheck, syscheck_entry_ac heck file

Aqui é possível notar coisas importantes como

Rule.id: 554

Rule.description: File added to the system

Syscheck.path: /root/malware.txt

Syscheck.event: added

Agente.name: ubuntu

Para compreender como o Wazuh identifica esse evento, é possível consultar a configuração da regra 554, localizada no arquivo 0015-ossec_rules.xml.

Essa regra é responsável por gerar o alerta “File added to the system” quando o FIM identifica a criação de um arquivo monitorado.

t rule.hipaa	164.312.c.1, 164.312.c.2
t rule.id	554
# rule.level	5
🔍 rule.mail	false

- Clique em 554 em rule.id

File added to the system.

[View alerts of this Rule](#)

Information

ID	Level	File	Path
554	5	0015-ossec_rules.xml	ruleset/rules
Groups syscheck, syscheck_entry_added, syscheck_file, ossec			

Details

Category	Decoded_as
ossec	syscheck_new_entry

Compliance

PCI DSS	GDPR 13	GDPR	HIPAA
554	File added to the system.	syscheck, syscheck_entry_added, syscheck_file, ossec	5
		0015-ossec_rules.xml	ruleset/rules
		syscheck. agentless.	

- Agora em 0015-ossec_rules.xml abaixo de File.

- Procure o id 554 nesse caso, depois em 0015-ossec_rules.xml

```
251 </rule>
252
253 <rule id="554" level="5">
254   <category>ossec</category>
255   <decoded_as>syscheck_new_entry</decoded_as>
256   <description>File added to the system.</description>
257   <group>syscheck, syscheck_entry_added, syscheck_file, pci_dss_11.5, gpg13_4.11, gdpr_II_5.1.f, hipaa_164.312.c.1, hipaa_164.312.c.2, nist_800_53_SI.7, tsc_PI1.4, tsc_PI1.5, tsc_CC6.1, tsc_CC6.8, tsc_CC7.2, tsc_CC7.3,</group>
258 </rule>
```

Ao achar o id da regra é possível analisá-la.

2) Detecção de intrusão em rede utilizando o Suricata IDS

O Suricata é uma ferramenta de detecção e prevenção de intrusões em rede (IDS/IPS), capaz de identificar atividades suspeitas por meio da análise do tráfego de rede.

Neste laboratório, o Suricata foi instalado na máquina Ubuntu e integrado ao Wazuh, permitindo centralizar os alertas de segurança gerados por ele.

O objetivo é simular uma varredura de portas utilizando o Kali Linux, verificar sua detecção pelo Suricata e realizar a contenção do endereço IP de origem utilizando o firewall pfSense manualmente.

A instalação do Suricata foi realizada com base na documentação disponível em:

<https://github.com/0xrajneesh/Suricata-IDS-Home-Lab/blob/main/installing-suricata.md>

OBS: Para organizar as regras de detecção do Suricata, foi necessário criar o diretório /etc/suricata/rules manualmente, onde são armazenados os arquivos de regras personalizados.

Os arquivos de regras devem possuir permissões que permitam sua leitura pelo processo do Suricata. Deve ser utilizada a permissão 644 no diretório “rules” para que o suricata consiga ler as regras.

Para integrar o Suricata ao Wazuh, foi configurada a coleta dos eventos registrados no arquivo /var/log/suricata/eve.json.

No arquivo /var/ossec/etc/ossec.conf do Wazuh Agent instalado no Ubuntu, foi adicionada uma entrada “localfile”, utilizando o formato json, permitindo que os eventos gerados pelo Suricata sejam encaminhados ao Wazuh Manager para análise e geração de alertas.

```
<localfile>
  <log_format>json</log_format>
  <location>/var/log/suricata/eve.json</location>
</localfile>
</ossec_config>
```

Depois é necessário reiniciar o agente Wazuh.

```
root@thiago-VMware-Virtual-Platform:/tmp# systemctl restart wazuh-agent
```

Para validar a detecção do Suricata, foi realizada uma varredura de portas utilizando o Nmap pelo Kali.

O objetivo foi simular uma atividade de reconhecimento de rede e verificar se o Suricata identificaria o tráfego suspeito, gerando alertas que seriam encaminhados ao Wazuh.

```
Nmap done: 1 IP address (1 host up) scanned in 0.68 seconds
(kali@Kali1)-[~]
$ nmap -sS 192.168.10.50
Starting Nmap 7.99 ( https://nmap.org ) at 2026-10-10 03:00 -0400
Nmap scan report for 192.168.10.50
Host is up (0.00021s latency).
Not shown: 999 closed tcp ports (reset)
PORT      STATE SERVICE
22/tcp    open  ssh
MAC Address: 00:0C:29:A4:1C:15 (VMware)
Nmap done: 1 IP address (1 host up) scanned in 0.68 seconds
```

Nesse caso o Kali detectou a porta 22 (ssh) aberta no ubuntu.

Logs do suricata:

```
root@thiago-VMware-Virtual-Platform:/etc/suricata/rules# cat /var/log/suricata/fast.log
10/10/2026-04:00:48.525367  [**] [1:2010937:3] ET SCAN Suspicious inbound to MySQL port 3306 [**] [Classification: Potentially Bad Traffic] [Priority: 2] {TCP} 192.168.10.200:46375 -> 192.168.10.50:3306
10/10/2026-04:00:48.537583  [**] [1:2010935:3] ET SCAN Suspicious inbound to MSSQL port 1433 [**] [Classification: Potentially Bad Traffic] [Priority: 2] {TCP} 192.168.10.200:46375 -> 192.168.10.50:1433
10/10/2026-04:00:48.538925  [**] [1:2010939:3] ET SCAN Suspicious inbound to PostgreSQL port 5432 [**] [Classification: Potentially Bad Traffic] [Priority: 2] {TCP} 192.168.10.200:46375 -> 192.168.10.50:5432
10/10/2026-04:00:48.543370  [**] [1:2002910:6] ET SCAN Potential VNC Scan 5800-5820 [**] [Classification: Attempted Information Leak] [Priority: 2] {TCP} 192.168.10.200:46375 -> 192.168.10.50:5800
10/10/2026-04:00:48.549040  [**] [1:2010936:3] ET SCAN Suspicious inbound to Oracle SQL port 1521 [**] [Classification: Potentially Bad Traffic] [Priority: 2] {TCP} 192.168.10.200:46375 -> 192.168.10.50:1521
root@thiago-VMware-Virtual-Platform:/etc/suricata/rules#
```

Alertas no Wazuh:

Oct 9, 2026 @ 04:01:11.203 - Oct 10, 2026 @ 04:01:11.203					
Export Formatted Reset view 714 available fields Columns Density 1 fields sorted Full screen					
	timestamp	agent.name	rule.description	rule.level	rule.id
	Oct 10, 2026 @ 04:00:50.148	ubuntu	Suricata: Alert - ET SCAN Suspicious inbound to Oracle SQL port...	3	86601
	Oct 10, 2026 @ 04:00:50.146	ubuntu	Suricata: Alert - ET SCAN Potential VNC Scan 5800-5820	3	86601
	Oct 10, 2026 @ 04:00:50.143	ubuntu	Suricata: Alert - ET SCAN Suspicious inbound to PostgreSQL por...	3	86601
	Oct 10, 2026 @ 04:00:50.141	ubuntu	Suricata: Alert - ET SCAN Suspicious inbound to MSSQL port 1433	3	86601
	Oct 10, 2026 @ 04:00:50.141	ubuntu	Suricata: Alert - ET SCAN Suspicious inbound to MySQL port 3306	3	86601

Ao expandir os detalhes do alerta é possível ver melhor o que está acontecendo:

data.alert.signature	ET SCAN Suspicious inbound to Oracle SQL port 1521
data.alert.signature_id	2010936
data.dest_ip	192.168.10.50
data.ip_v	4
data.pkt_src	wire/pcap
data.proto	TCP
data.src_ip	192.168.100.200
data.src_port	63055

Alert.signature: ET SCAN Suspicious inbound to Oracle SQL port 1521

Rule.id: 86601

Src_ip e Dst_ip: 192.168.100.200/192.168.10.50

Dest_port: 1521

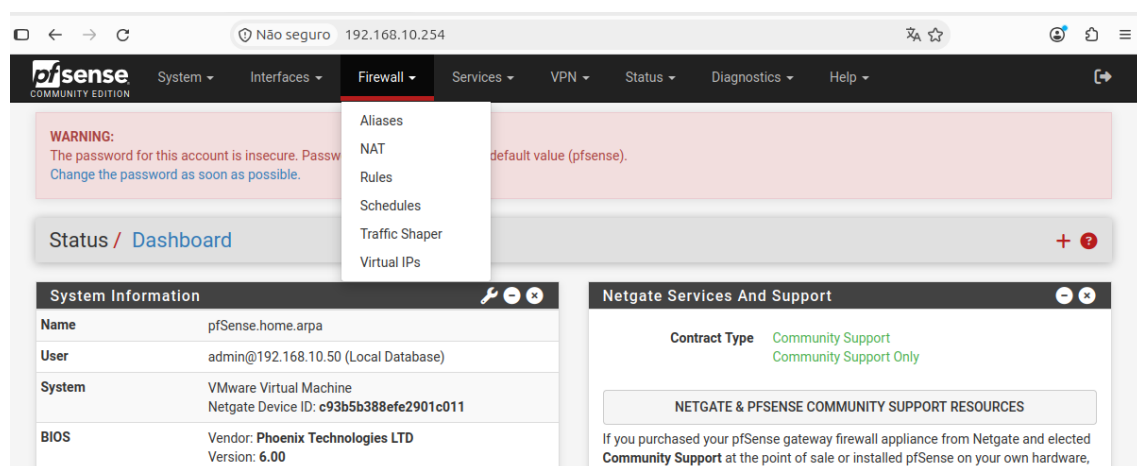
Data.alert.severity: 2

A análise do alerta identificou uma atividade suspeita originada do IP 192.168.100.200 com destino ao host 192.168.10.50, envolvendo a porta 1521, normalmente utilizada pelo Oracle Database.

O Suricata acionou a assinatura ET SCAN Suspicious inbound to Oracle SQL port 1521 com severidade 2.

Após a identificação da atividade suspeita, foi realizada uma ação de contenção utilizando o firewall pfSense manualmente.

Na interface do pfSense, Firewall → rules pode-se criar uma regra de tráfego.



Edit Firewall Rule

Action Block
 Choose what to do with packets that match the criteria specified below.
 Hint: the difference between block and reject is that with reject, a packet (TCP RST or ICMP port unreachable for UDP) is returned to the sender, whereas with block the packet is dropped silently. In either case, the original packet is discarded.

Source
Source ☐ Invert match Address or Alias 192.168.100.200 /

Destination
Destination ☐ Invert match Network 192.168.10.0 / 24

Extra Options
Log ☐ Log packets that are handled by this rule
 Hint: the firewall has limited local log space. Don't turn on logging for everything. If doing a lot of logging, consider using a remote syslog server (see the [Status: System Logs: Settings](#) page).

Description IP malicioso bloqueado
 A description may be entered here for administrative reference. A maximum of 57 characters will be used in the ruleset label and displayed in the firewall log.

Advanced Options Display Advanced

Foi criada uma regra de bloqueio para impedir que o endereço IP 192.168.100.200 (Kali Linux) estabelecesse comunicação com a rede 192.168.10.0/24, onde está localizado o host monitorado.

O objetivo foi simular uma ação de resposta a incidentes, impedindo novas conexões originadas da máquina atacante.

```
(kali@Kali1)~[~]
$ ping 192.168.10.50
PING 192.168.10.50 (192.168.10.50) 56(84) bytes of data.
^C
— 192.168.10.50 ping statistics —
3 packets transmitted, 0 received, 100% packet loss, time 2049ms
```

Após a aplicação da regra no pfSense, foi realizado um teste de conectividade utilizando o comando ping a partir do Kali Linux para o endereço 192.168.10.50.

O teste apresentou 100% de perda de pacotes, mostrando o bloqueio do tráfego pela regra configurada no firewall.

3) Detectando Vulnerabilidades

O Wazuh possui um módulo chamado Vulnerability Detection, responsável por identificar vulnerabilidades conhecidas (CVEs) em sistemas monitorados, utilizando informações de inventário de softwares e dados de vulnerabilidades.

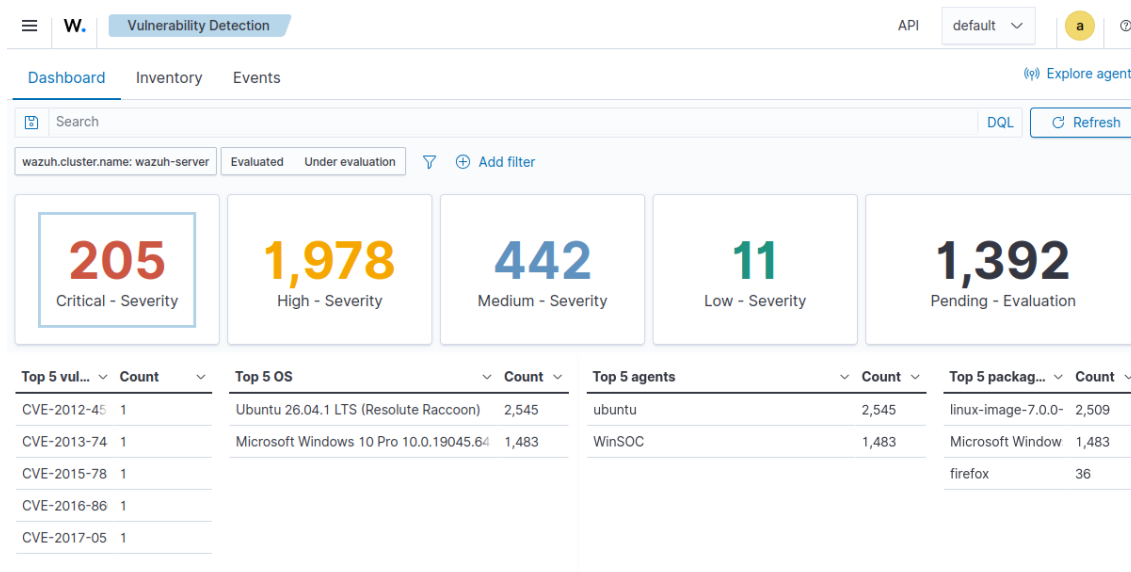
Neste laboratório, o objetivo é identificar vulnerabilidades nos endpoints monitorados e consultar suas informações para auxiliar na avaliação de riscos e na priorização de possíveis ações de mitigação.

Para habilitar essa funcionalidade, é necessário verificar a configuração do módulo vulnerability-detection no arquivo /var/ossec/etc/ossec.conf do Wazuh Manager.

```
<vulnerability-detection>
  <enabled>yes</enabled>
  <index-status>yes</index-status>
  <feed-update-interval>60m</feed-update-interval>
</vulnerability-detection>
```

No parâmetro enabled, o valor yes indica que o módulo de detecção de vulnerabilidades está habilitado no Wazuh Manager.

Após habilitar o módulo, é possível acessar Threat Intelligence > Vulnerability Detection no dashboard do Wazuh para visualizar as vulnerabilidades identificadas nos endpoints monitorados.



Na aba Inventory, é possível consultar individualmente as vulnerabilidades identificadas pelo Wazuh, verificando informações como o identificador CVE, o nível de severidade e o endpoint afetado.

agent.name	package.name	package.version	vulnerability.descripti...	vulnerability.severity	vulnerability.id
WinSOC	Microsoft Windows 10 Pr...	10.0.19045.6456	Heap-based buffer overfl...	High	CVE-2026-85880
WinSOC	Microsoft Windows 10 Pr...	10.0.19045.6456	Heap-based buffer overfl...	High	CVE-2026-83988
WinSOC	Microsoft Windows 10 Pr...	10.0.19045.6456	Heap-based buffer overfl...	High	CVE-2026-83987
WinSOC	Microsoft Windows 10 Pr...	10.0.19045.6456	Heap-based buffer overfl...	High	CVE-2026-83986
WinSOC	Microsoft Windows 10 Pr...	10.0.19045.6456	Heap-based buffer overfl...	High	CVE-2026-83985
WinSOC	Microsoft Windows 10 Pr...	10.0.19045.6456	Heap-based buffer overfl...	High	CVE-2026-83982
WinSOC	Microsoft Windows 10 Pr...	10.0.19045.6456	Heap-based buffer overfl...	High	CVE-2026-83981
WinSOC	Microsoft Windows 10 Pr...	10.0.19045.6456	Heap-based buffer overfl...	High	CVE-2026-83980
WinSOC	Microsoft Windows 10 Pr...	10.0.19045.6456	Use after free in Window...	High	CVE-2026-83979
WinSOC	Microsoft Windows 10 Pr...	10.0.19045.6456	Heap-based buffer overfl...	High	CVE-2026-83976

Ao expandir os detalhes de uma vulnerabilidade, é possível consultar informações adicionais sobre a CVE identificada, permitindo compreender qual componente está afetado e avaliar sua gravidade.

vulnerability.description	Heap-based buffer overflow in Windows ALPC allows an authorized attacker to elevate privileges locally.
vulnerability.detected_at	Oct 9, 2026 @ 18:29:32.989
vulnerability.enumeration	CVE
vulnerability.id	CVE-2026-85880
vulnerability.published_at	Sep 8, 2026 @ 15:21:14.000
vulnerability.reference	https://msrc.microsoft.com/update-guide/vulnerability/CVE-2026-85880 , https://www.cisa.gov/known-exploited-vulnerabilities-catalog?field_cve=CVE-2026-85880
vulnerability.scanner.condition	Package less than 10.0.19045.7725
vulnerability.scanner.reference	https://cti.wazuh.com/vulnerabilities/cves/CVE-2026-85880

**CVE-2026-85880**

Published2026/9/8NVD

Heap-based buffer overflow in Windows ALPC allows an authorized attacker to elevate privileges locally.

7.8

High

010

NVD

Attack ComplexityLOW

Availability ImpactHIGH

Confidentiality ImpactHIGH

Integrity ImpactHIGH

Attack VectorLocal

Durante a análise de uma das vulnerabilidades apresentadas, foi identificada a CVE-2026-85880, relacionada a um Heap-based Buffer Overflow no Windows ALPC, com severidade alta (CVSS 7.8).

Essa falha pode permitir que um atacante com acesso local eleve seus privilégios no sistema. O Wazuh identificou uma versão do pacote inferior a 10.0.19045.7725, indicando a necessidade de verificar as atualizações de segurança aplicáveis.

Como medida de mitigação, recomenda-se aplicar as atualizações de segurança disponibilizadas pela Microsoft e realizar uma nova verificação para confirmar a remediação.

4) Detecção da execução de comandos maliciosos

O auditd é um serviço de auditoria do Linux que permite registrar atividades do sistema, como execução de comandos, acesso a arquivos e alterações de permissões, conforme as regras de auditoria configuradas.

Neste laboratório, o objetivo é integrar os registros do auditd ao Wazuh para identificar atividades potencialmente suspeitas, como acesso a arquivos sensíveis e execução de comandos administrativos.

Os eventos coletados pelo auditd são armazenados no arquivo `/var/log/audit/audit.log`, que será utilizado como fonte de logs para o Wazuh.

```
thiago@thiago-VMware-Virtual-Platform:/var/log$ sudo apt install auditd -y
[sudo: authenticate] Senha:
Atualizando:
  libaudit-common  libaudit1

Instalando:
  auditd
```

Para integrar o auditd ao Wazuh, foi configurada a coleta do arquivo `/var/log/audit/audit.log` no Wazuh Agent instalado no Ubuntu.

No arquivo `/var/ossec/etc/ossec.conf`, foi adicionada uma entrada `<localfile>`, utilizando o formato audit, permitindo que os registros de auditoria sejam encaminhados ao Wazuh Manager para análise e geração de alertas.

```
<localfile>
  <log_format>audit</log_format>
  <location>/var/log/audit/audit.log</location>
</localfile>
</ossec_config>
```

Reinicie o serviço do wazuh-agent com “**systemctl restart wazuh-agent**”

O auditd permite configurar regras de auditoria para monitorar atividades específicas do sistema em `/etc/audit/audit.rules`

```
root@thiago-VMware-Virtual-Platform: /var/log/audi...
/var/log/audit

## This file is automatically generated from /etc/audit/rules.d
-D
-b 8192
-f 1
--backlog_wait_time 60000

~
~
~
```

Nesse laboratório foram criadas 2, uma para registrar o acesso ao arquivo `/etc/passwd` e outra para a execução do comando `useradd`, permitindo acompanhar atividades relacionadas à consulta de contas e à criação de usuários no Linux.

```
## This file is automatically generated from /etc/audit/rules.d
-D
-b 8192
-f 1
--backlog_wait_time 60000

-w /etc/passwd -p r -k audit-wazuh-r
-a always,exit -F arch=b64 -S execve -F path=/usr/sbin/useradd -F perm=x -k audit-wazuh-x
```

O parâmetro -k permite atribuir uma chave de identificação aos eventos gerados pelas regras do auditd, facilitando sua pesquisa e correlação nos logs.

Essas chaves podem ser utilizadas pelo Wazuh na criação de regras personalizadas de detecção. O Wazuh também oferece CDB Lists, que permitem consultar listas de valores durante a avaliação de eventos, mas não é obrigatório que a chave -k corresponda a uma entrada nessas listas.

The screenshot shows the Wazuh CDB Lists interface. At the top, there's a header with the Wazuh logo, a 'CDB Lists' tab, and an 'API' dropdown set to 'default'. Below the header, there's a section for 'audit-keys' with a search bar and a table of keys. The table has columns for 'Key', 'Value', and 'Actions'. The keys listed are 'audit-wazuh-w' (write), 'audit-wazuh-r' (read), 'audit-wazuh-a' (attribute), 'audit-wazuh-x' (execute), and 'audit-wazuh-c' (command). Each key has a corresponding value and a set of actions (edit and delete icons). At the bottom, there's a 'Rows per page' dropdown set to 10 and a pagination control showing page 1 of 1.

Key	Value	Actions
audit-wazuh-w	write	
audit-wazuh-r	read	
audit-wazuh-a	attribute	
audit-wazuh-x	execute	
audit-wazuh-c	command	

Após configurar as regras de auditoria, é necessário carregá-las para que o auditd passe a monitorar as regras definidas.

```
root@thiago-VMware-Virtual-Platform:/var/log/audit# auditctl -R /etc/audit/audit.rules
```

Para validar as regras de auditoria configuradas, foram realizados testes no Ubuntu, envolvendo o acesso ao arquivo /etc/passwd e a execução do comando useradd.

O objetivo foi verificar se o auditd registraria essas atividades e se os eventos seriam encaminhados ao Wazuh para análise.

```
root@thiago-VMware-Virtual-Platform:/var/log/audit# cat /etc/passwd
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
sys:x:3:3:sys:/dev:/usr/sbin/nologin
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/usr/sbin/nologin
man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin
mail:x:8:8:mail:/var/mail:/usr/sbin/nologin
news:x:9:9:news:/var/spool/news:/usr/sbin/nologin
uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin
proxy:x:13:13:proxy:/bin:/usr/sbin/nologin
www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin
backup:x:34:34:backup:/var/backups:/usr/sbin/nologin

root@thiago-VMware-Virtual-Platform:/var/log/audit# useradd hacker
root@thiago-VMware-Virtual-Platform:/var/log/audit# id hacker
uid=1001(hacker) gid=1001(hacker) groups=1001(hacker)
```

W.	Threat Hunting	API	default	a	?
3,133 hits					
Oct 9, 2026 @ 15:12:00.447 - Oct 10, 2026 @ 15:12:00.447					
Export Formatted Reset view 764 available fields Columns Density 1 fields sorted Full screen					
timestamp	agent.name	rule.description	rule.level	rule.id	
Oct 10, 2026 @ 15:11:30.023	ubuntu	Audit: Watch - Execute access: /usr/sbin/useradd.	3	80789	
Oct 10, 2026 @ 15:10:58.005	ubuntu	Audit: Watch - Read access: /etc/passwd.	3	80784	

Após a execução dos testes, os eventos foram registrados pelo auditd e encaminhados ao Wazuh, confirmando que a integração está funcionando.

Os alertas gerados podem ser consultados no dashboard do Wazuh, permitindo investigar as atividades monitoradas pelas regras de auditoria.

Table	JSON
_index	wazuh-alerts-4.x-2026.10.10
agent.id	005
agent.ip	192.168.10.50
agent.name	ubuntu
data.audit.arch	c000003e
data.audit.auid	1000
data.audit.command	cat
data.audit.cwd	/var/log/audit
data.audit.exe	/usr/lib/cargo/bin/coreutils/cat
data.audit.exit	3
data.audit.file.inode	133869
data.audit.file.mode	0100644
data.audit.file.name	/etc/passwd

Document Details

[View surrounding documents](#)

[View single document](#)

Table JSON

<code>_index</code>	wazuh-alerts-4.x-2026.10.10
<code>agent.id</code>	005
<code>agent.ip</code>	192.168.10.50
<code>agent.name</code>	ubuntu
<code>data.dstuser</code>	hacker
<code>data.gid</code>	1001
<code>data.home</code>	/home/hacker
<code>data.shell</code>	/bin/sh,
<code>data.uid</code>	1001
<code>decoder.name</code>	useradd
<code>decoder.parent</code>	useradd
<code>full_log</code>	Oct 10 18:11:55 thiago-VMware-Virtual-Platform useradd[12472]: new user: name=hacker, UID=1001, GID=1001, home=/home/hacker, shell=/bin/sh, from=/dev/pts/1
<code>id</code>	1791655916.1701816

A análise do evento identificou a execução do comando `cat`, associado ao acesso ao arquivo `/etc/passwd`, no host Ubuntu (192.168.10.50)

Também identificou a criação do usuário “hacker” no mesmo host logo em seguida.

O monitoramento de atividades como acesso a arquivos sensíveis, criação de usuários e alterações em mecanismos de persistência é importante para identificar possíveis comportamentos suspeitos.

No Linux, comandos como `useradd`, `crontab` e `sudo -l` podem estar associados à criação de contas, persistência e reconhecimento de privilégios.

5) Detectando e bloqueando ataques de força bruta SSH

O Wazuh possui regras nativas capazes de identificar tentativas de autenticação malsucedidas no SSH, incluindo ataques de força bruta.

Neste laboratório, o objetivo é simular um ataque de força bruta SSH a partir do Kali e utilizar o recurso Active Response do Wazuh para bloquear automaticamente o endereço IP de origem após a detecção.

O Active Response permite executar ações automatizadas quando determinadas regras de detecção são acionadas. Neste cenário, será utilizado o comando `firewall-drop`, que utiliza um script para adicionar uma regra de bloqueio no firewall local do Ubuntu.

É necessário criar a regra no servidor Wazuh em `/var/ossec/etc/ossec.conf`

```
<!-- Active response -->
<global>
  <white_list>127.0.0.1</white_list>
  <white_list>localhost.localdomain</white_list>
  <white_list>172.31.0.2</white_list>
  <white_list>fd17:625c:f037:2::3</white_list>
</global>

<command>
  <name>disable-account</name>
  <executable>disable-account</executable>
  <timeout_allowed>yes</timeout_allowed>
</command>

<command>
  <name>restart-wazuh</name>
  <executable>restart-wazuh</executable>
</command>

<command>
  <name>firewall-drop</name>
  <executable>firewall-drop</executable>
  <timeout_allowed>yes</timeout_allowed>
</command>
```

Antes de configurar o Active Response, é necessário definir o comando que será executado pelo Wazuh.

Neste laboratório, foi utilizado o `firewall-drop`, um comando já disponibilizado pelo Wazuh, que permite bloquear endereços IP por meio do firewall local.

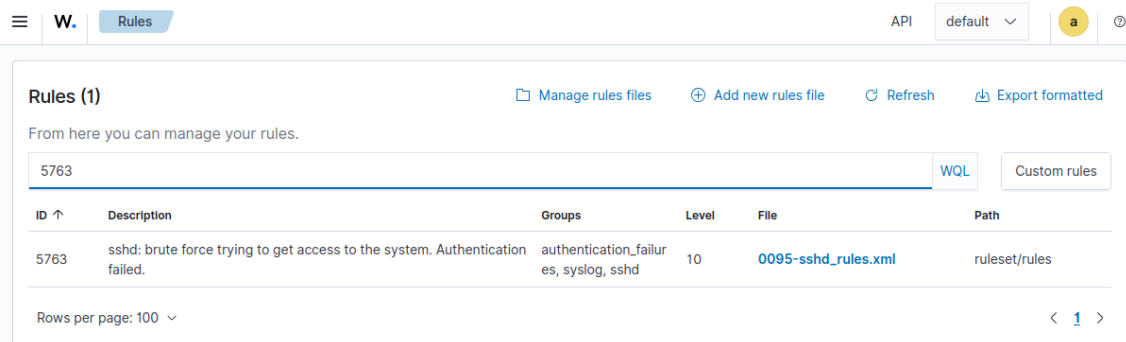
Após definir o comando, a seção `active-response` permite configurar quais regras de detecção irão acioná-lo, onde a resposta será executada e por quanto tempo o bloqueio permanecerá ativo.

```
<!--
<active-response>
  active-response options here
</active-response>
-->

<active-response>
  <command>firewall-drop</command>
  <location>local</location>
  <rules_id>5763</rules_id>
  <timeout>180</timeout>
</active-response>
```

Na seção `active-response`, o comando `firewall-drop` foi associado à regra 5763 do Wazuh, relacionada à detecção de múltiplas falhas de autenticação SSH.

Quando essa regra é acionada, o Wazuh executa a resposta automatizada no endpoint monitorado, adicionando um bloqueio para o endereço IP de origem no firewall local.



Rules (1)

From here you can manage your rules.

ID ↑	Description	Groups	Level	File	Path
5763	sshd: brute force trying to get access to the system. Authentication failed.	authentication_failures, syslog, sshd	10	0095-sshd_rules.xml	ruleset/rules

Rows per page: 100

Para compreender o funcionamento da regra 5763, foi acessada a seção `Server Management > Rules` no dashboard do Wazuh.

Nessa seção, é possível consultar a descrição da regra, seu nível de severidade e as condições necessárias para que o alerta seja gerado.

Para validar a detecção e a resposta automatizada, foi realizado um ataque de força bruta contra o serviço SSH do Ubuntu, usando a ferramenta Hydra no do Kali.

O objetivo foi gerar múltiplas tentativas de autenticação malsucedidas, verificar o acionamento da regra 5763 e confirmar a execução do firewall-drop pelo Wazuh.

```
(kali@kali)-[/usr/share/wordlists]
└─$ hydra -l thiago -p senhas.txt ssh://192.168.10.50 -t 4
Hydra v9.7 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organizations, or for illegal purposes (this is non-binding, these ** ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-10-10 15:31:12
[DATA] max 4 tasks per 1 server, overall 4 tasks, 3560 login tries (l:1/p:3560), ~890 tries per task
[DATA] attacking ssh://192.168.10.50:22/
```

W. Threat Hunting API default a

7,066 hits

Oct 9, 2026 @ 16:32:25.002 - Oct 10, 2026 @ 16:32:25.003

Export Formatted Reset view 772 available fields Columns Density 1 fields sorted Full screen

timestamp	agent.name	rule.description	rule.level	rule.id
Oct 10, 2026 @ 16:32:09.070	ubuntu	sshd: authentication failed.	5	5760
Oct 10, 2026 @ 16:32:08.604	ubuntu	Host Blocked by firewall-drop Active Response	3	651
Oct 10, 2026 @ 16:32:07.074	ubuntu	sshd: brute force trying to get access to the system. Authenticat...	10	5763
Oct 10, 2026 @ 16:32:07.071	ubuntu	sshd: authentication failed.	5	5760
Oct 10, 2026 @ 16:32:07.069	ubuntu	sshd: authentication failed.	5	5760
Oct 10, 2026 @ 16:32:07.069	ubuntu	unix_chkpwd: Password check failed.	5	5557
Oct 10, 2026 @ 16:32:05.073	ubuntu	sshd: authentication failed.	5	5760
Oct 10, 2026 @ 16:32:05.071	ubuntu	unix_chkpwd: Password check failed.	5	5557

Após a execução das tentativas de força bruta SSH, o Wazuh gerou um alerta e acionou o Active Response configurado para bloquear o endereço IP de origem.

O alerta indica que o mecanismo de detecção e resposta automatizada foi acionado durante o teste.

```
thiago@thiago-VMware-Virtual-Platform:~$ sudo iptables -L -n -v
Chain INPUT (policy ACCEPT 1137 packets, 142K bytes)
 pkts bytes target     prot opt in     out     source            destination
   76 8784 DROP       all  --  *      *       192.168.100.200    0.0.0.0/0

Chain FORWARD (policy ACCEPT 0 packets, 0 bytes)
 pkts bytes target     prot opt in     out     source            destination
    0    0 DROP       all  --  *      *       192.168.100.200    0.0.0.0/0
```

Para validar a execução do Active Response, foi consultada a tabela de regras do iptables no Ubuntu. Foi possível identificar a regra de bloqueio adicionada pelo firewall-drop, confirmando que o Wazuh executou a ação de contenção configurada para o endereço IP de origem.

data.parameters.alert.agen	192.168.10.50
t.ip	
data.parameters.alert.agen	ubuntu
t.name	
data.parameters.alert.data.d	thiago
stuser	
data.parameters.alert.data.s	192.168.100.200
rcip	
data.parameters.alert.data.s	53232
rcport	

①	data.parameters.alert.rule.description	sshd: brute force trying to get access to the system. Authentication failed.
②	data.parameters.alert.rule.firedtimes	3
③	data.parameters.alert.rule.frequency	8
④	data.parameters.alert.rule.groups	IV_35.7.d, IV_32.2 dpr
⑤	data.parameters.alert.rule.groups	syslog, sshd, authentication_failures

Ao analisar os detalhes do alerta no Wazuh, é possível identificar o endereço IP de origem das tentativas de autenticação, o host afetado e a regra de detecção acionada.

6) Detecção de arquivos maliciosos usando o VirusTotal

O Wazuh permite integrar o VirusTotal para consultar a reputação de arquivos por meio de seus hashes, auxiliando na identificação de possíveis ameaças.

Neste laboratório, o objetivo é integrar o VirusTotal ao File Integrity Monitoring (FIM) do Wazuh para verificar arquivos criados ou modificados em um diretório monitorado.

Para validar a integração, será utilizado o arquivo de teste EICAR, desenvolvido para simular uma detecção de antivírus sem utilizar malware real.

```
<!-- File integrity monitoring -->
<syscheck>
<disabled>no</disabled>

<!-- Frequency that syscheck is executed default every 12 hours -->
<frequency>43200</frequency>

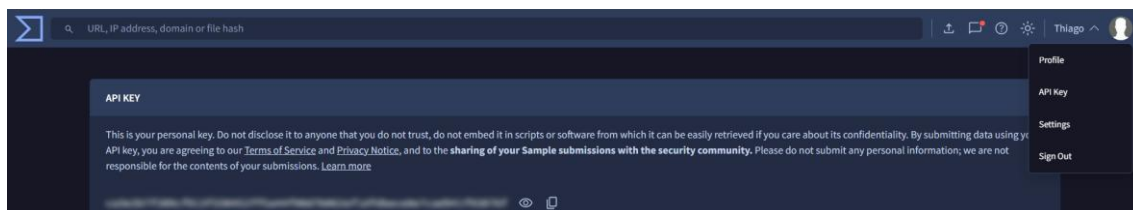
<scan_on_start>yes</scan_on_start>

<!-- Directories to check (perform all possible verifications) -->
<directories>/etc,/usr/bin,/usr/sbin</directories>
<directories>bin,/sbin,/boot</directories>
<directories check_all="yes" report_changes="yes" realtime="yes">/root</directories>
```

Neste cenário, será utilizado novamente o diretório /root, configurado anteriormente no FIM para monitorar a criação e modificação de arquivos que serão submetidos à consulta de reputação no VirusTotal.

Para integrar o VirusTotal ao Wazuh, foi adicionada uma configuração na seção integration do arquivo /var/ossec/etc/ossec.conf, localizado no Wazuh Manager.

Essa integração permite que o Wazuh consulte a API do VirusTotal utilizando os hashes dos arquivos monitorados pelo FIM, obtendo informações sobre possíveis detecções por mecanismos de segurança.



```

<!-- Configuration for wazuh-authd -->
<auth>
  <disabled>no</disabled>
  <port>1515</port>
  <use_source_ip>no</use_source_ip>
  <purge>yes</purge>
  <use_password>no</use_password>
  <ciphers>HIGH:!ADH:!EXP:!MD5:!RC4:!3DES:!CAMELLIA:@STRENGTH</ciphers>
  <!-- <ssl_agent_ca></ssl_agent_ca> -->
  <ssl_verify_host>no</ssl_verify_host>
  <ssl_manager_cert>etc/sslmanager.cert</ssl_manager_cert>
  <ssl_manager_key>etc/sslmanager.key</ssl_manager_key>
  <ssl_auto_negotiate>no</ssl_auto_negotiate>
</auth>

<cluster>
  <name>wazuh</name>
  <node_name>node01</node_name>
  <node_type>master</node_type>
  <key></key>
  <port>1516</port>
  <bind_addr>0.0.0.0</bind_addr>
  <nodes>
    <node>NODE_IP</node>
  </nodes>
  <hidden>no</hidden>
  <disabled>yes</disabled>
</cluster>

<integration>
  <name>virustotal</name>
  <api_key>[REDACTED]</api_key>
  <rule_id>550,554</rule_id>
  <alert_format>json</alert_format>
</integration>
/ossec_config

```

Para habilitar a integração, foi utilizada uma chave de API do VirusTotal, configurada no parâmetro `api_key` do arquivo `ossec.conf` no Wazuh Manager.

Após salvar as alterações, foi reiniciado o serviço `wazuh-manager` para aplicar a configuração.

No parâmetro `rule_id`, foram configuradas as regras 550 e 554 do Wazuh, relacionadas à modificação e à criação de arquivos, respectivamente.

Quando uma dessas regras é acionada pelo FIM, a integração permite consultar o hash do arquivo no VirusTotal para verificar possíveis detecções.

```

<rule id="550" level="7">
  <category>ossec</category>
  <decoded_as>syscheck_integrity_changed</decoded_as>
  <description>Integrity checksum changed.</description>
  <mitre>
    <id>T1565.001</id>
  </mitre>
  <group>syscheck, syscheck_entry_modified, syscheck_file, pci_dss_11.5, gpg13_4.11, gdpr_II_5.1.f, hipaa_164.312.c.1, hipaa_164.312.c.2, nist_800_53_SI.7, tsc_PI1.4, tsc_PI1.5, tsc_CC6.1, tsc_CC6.8, tsc_CC7.2, tsc_CC7.3,</group>
</rule>

<rule id="554" level="5">
  <category>ossec</category>
  <decoded_as>syscheck_new_entry</decoded_as>
  <description>File added to the system.</description>
  <group>syscheck, syscheck_entry_added, syscheck_file, pci_dss_11.5, gpg13_4.11, gdpr_II_5.1.f, hipaa_164.312.c.1, hipaa_164.312.c.2, nist_800_53_SI.7, tsc_PI1.4, tsc_PI1.5, tsc_CC6.1, tsc_CC6.8, tsc_CC7.2, tsc_CC7.3,</group>
</rule>

```

As duas equivalem a criação e modificação de arquivos, caso uma delas for acionada o virustotal irá verificar o arquivo

```

root@thiago-VMware-Virtual-Platform:~# touch teste.txt
root@thiago-VMware-Virtual-Platform:~# pwd
/root
root@thiago-VMware-Virtual-Platform:~#

```

Oct 10, 2026 @ 17:25:39.084	ubuntu	Suricata: Alert - GPL ICMP_INFO PING *NIX	3	86601
Oct 10, 2026 @ 17:25:37.998	ubuntu	VirusTotal: Alert - /root/teste.txt - No positives found	3	87104
Oct 10, 2026 @ 17:25:37.083	ubuntu	Suricata: Alert - GPL ICMP_INFO PING *NIX	3	86601

Após configurar a integração, foi realizado um teste para verificar a comunicação entre o Wazuh e a API do VirusTotal.

O resultado apresentado no dashboard confirmou que a consulta foi processada, demonstrando o funcionamento com o virustotal.

Para validar a detecção, foi utilizado o arquivo de teste EICAR, desenvolvido para verificar o funcionamento de soluções antimalware sem a necessidade de executar um malware real.

O arquivo será utilizado para testar se o Wazuh consegue consultar seu hash no VirusTotal e apresentar os resultados da análise de reputação.



× 🗨️ 🔊 🔄 ⚙️ 🔗 📱 Fazer login

Modo IA **Tudo** Imagens Vídeos Notícias Shopping Vídeos curtos Mais Ferramentas

Visão geral criada por IA

The **EICAR test file** is a harmless, 68-byte computer file used to safely test whether antivirus and antimalware software detect threats properly without using real malware.



What is the EICAR String?

- Standard String:** X50!P%AP[4\ZX54(P^)7CC)7)\$EICAR-STANDARD-ANTIVIRUS-TEST-FILE!\$H+H*

Nature: It contains a specific printable ASCII character sequence. It does not contain

Mostrar mais



eicar
https://www.eicar.org/download-anti-malware-testfile/

Download Anti Malware Testfile

Instead of using real malware, which could cause real damage, this test file allows people to test anti-virus software without having to use a real...



EICAR (EICAR test file)

Arquivo de teste

O EICAR test file é um arquivo de 68






www.eicar.org/download-anti-malware-testfile/





+49 8194 99 84 99 CONTACT



EICAR ABOUT TEAM NEWS PROJECTS MEMBERSHIP

DOWNLOAD ANTI MALWARE TESTFILE


DOWNLOAD AREA

using the secure, SSL enabled protocol HTTPS

eicar.com

Com-file

68 Bytes

DOWNLOAD

eicar.com.txt

1 Text-file

68 Bytes

DOWNLOAD

eicar.com.zip

1 Zip-file

184 Bytes

DOWNLOAD

eicar.com-2.zip

1 Zip-file

308 Bytes

DOWNLOAD

Após o download, o arquivo EICAR foi movido para o diretório /root.

A movimentação do arquivo para esse diretório teve como objetivo gerar um evento de criação, acionando a regra 554 do Wazuh e permitindo a consulta de seu hash no VirusTotal.

```
thiago@thiago-VMware-Virtual-Platform:~$ sudo mv eicar.com.txt /root
[sudo: authenticate] Senha:
thiago@thiago-VMware-Virtual-Platform:~$
```

	Oct 10, 2026 @ 17:31:47.241	ubuntu	Suricata: Alert - GPL ICMP_INFO PING *NIX	3	86601
	Oct 10, 2026 @ 17:31:46.133	ubuntu	VirusTotal: Alert - /root/eicar.com.txt - 66 engines detected this ...	12	87105
	Oct 10, 2026 @ 17:31:45.497	ubuntu	PAM: Login session closed.	3	5502

Após a consulta do hash do arquivo EICAR ao VirusTotal, 66 mecanismos de segurança apresentaram resultados positivos para detecção.

Esse resultado confirma que a integração conseguiu consultar a reputação do arquivo e retornar as informações ao Wazuh.

Como o EICAR é um arquivo de teste, essas detecções são esperadas e não representam a identificação de um malware real.

t	_index	wazuh-alerts-4.x-2026.10.10
t	agent.id	005
t	agent.ip	192.168.10.50
t	agent.name	ubuntu
t	data.integration	virustotal
t	rule.description	VirusTotal: Alert - /root/eicar.com.txt - 66 engines detected this file
#	rule.firedtimes	1
t	rule.gdpr	IV_35.7.d
t	rule.groups	virustotal
t	rule.id	87105
#	rule.level	12
	rule.mail	true
t	rule.mitre.id	T1203
t	rule.mitre.tactic	Execution
t	rule.mitre.technique	Exploitation for Client Execution

Ao analisar os detalhes do alerta no Wazuh, é possível identificar o host monitorado, a regra de detecção acionada e as informações retornadas pela integração com o VirusTotal.

O Wazuh também pode apresentar referências ao framework MITRE ATT&CK associadas à regra.